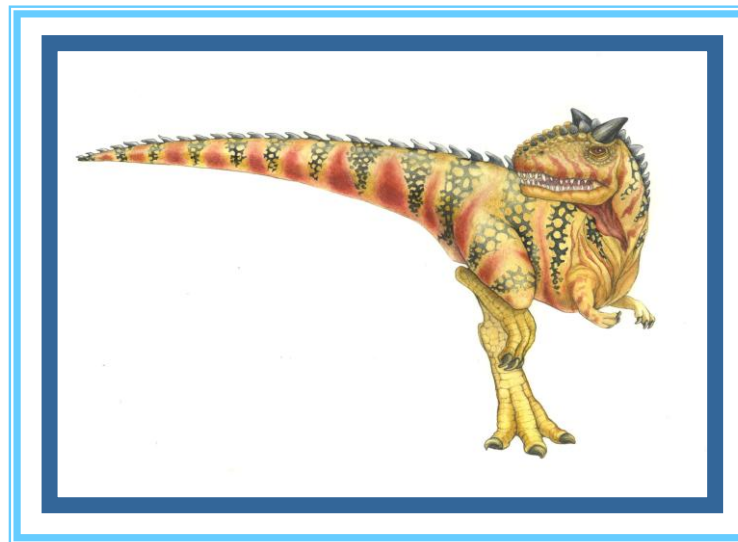


Unit-5 Part-1:

Disk Management





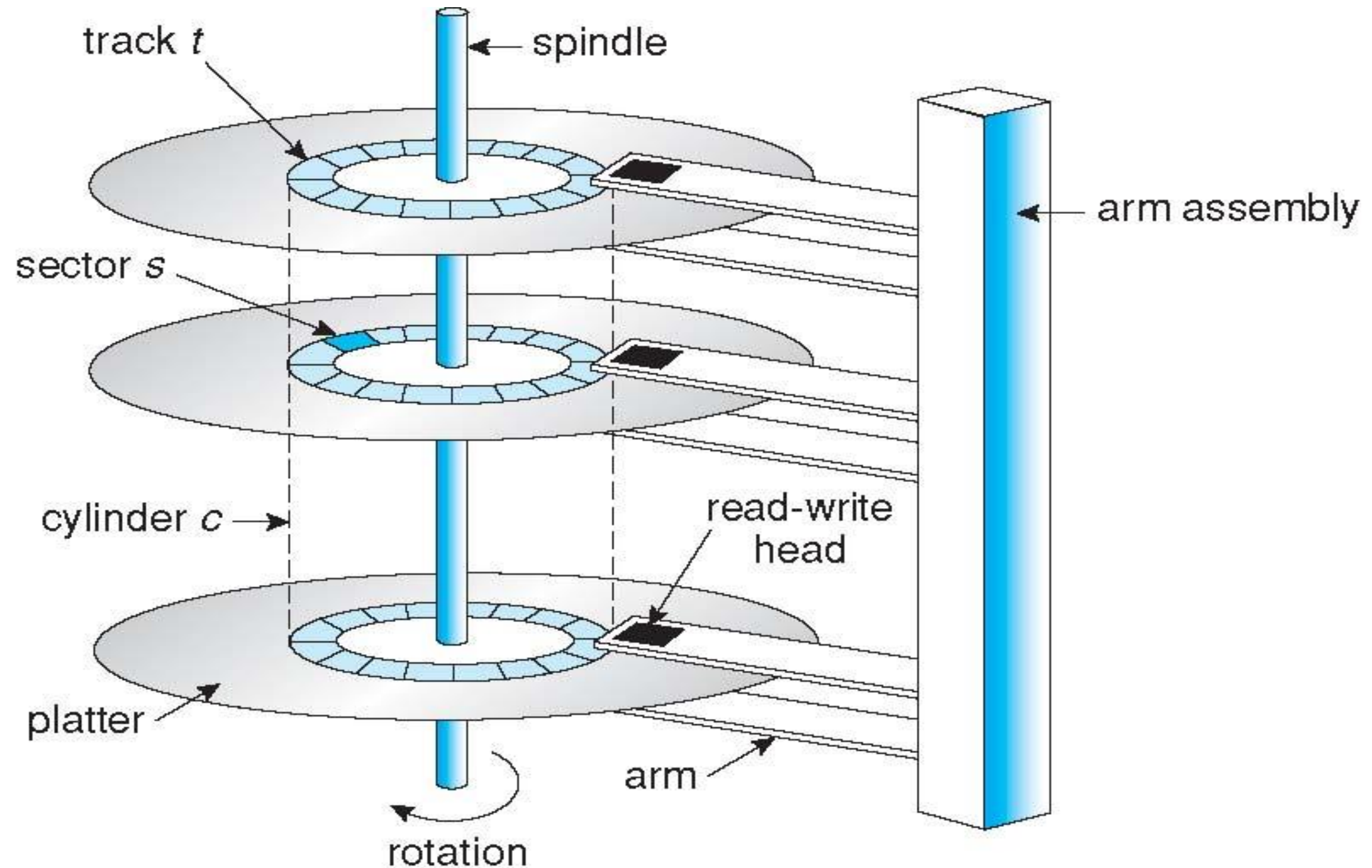
Overview of Magnetic Disk

- **Magnetic disks** provide bulk of secondary storage of modern computers
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer.
 - **Positioning time** (**random-access time**) is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**).
- Disks can be removable
- Drive attached to computer via **I/O bus**
 - Busses vary, including **ATA, SATA, USB, Fibre Channel etc.**
 - **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array





Moving-head Disk Mechanism





Disk Scheduling

- Efficiency is a major concern while accessing any I/O device due to speed mismatch between the processor and the devices.
- This chapter discusses some efficiency and performance issues of the disk as it is a widely used I/O device.
- Disk scheduling is one of the major parts of disk management that schedules the I/O requests arriving on the disk.
- This chapter explains various disk-scheduling algorithms, disk formatting, and bad sectors. The discussion extends to swap-space management and RAID structures.





Disk Scheduling...

- ❑ Disk scheduling schedules the I/O requests arriving on the disk. There are several reasons disk scheduling is important for I/O operations.
- ❑ In a multi-programming environment, many processes may send I/O requests for the disk. Since the processor can service only one I/O request at a time, others may need to wait on the disk queue. When the current request is finished, another request from the disk queue is scheduled to be serviced.
- ❑ While accessing a particular location on the disk, the current position of the disk arm matters. If the next request is far from the current position, then the seek time is more as compared to other requests having less seek time.





Disk Scheduling...

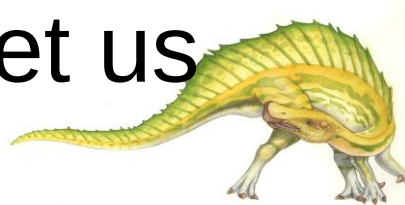
- ❑ Random requests from the users may also need disk scheduling so that various types of requests of different users are serviced.
- ❑ For example, in case of web servers and data base servers, the users may request smaller to larger data in size.
- ❑ Moreover, the requests may not be on the same cylinder. Keeping in view such facts, there is a need to schedule the requests on the disk to improve the performance of the system. In this direction, various scheduling algorithms have been developed.





DISK-SCHEDULING CRITERIA

- **Disk I/O operation** is a major aspect of the computer system.
- Since it is dependent on the movement of the disk arm, the disk scheduling is performed such that the arm movement is utilized to service more I/O requests efficiently.
- For accessing the disk, the logical blocks are converted into the disk physical blocks. However, the disk physical block address is in the form of cylinder, track, and sector: **First, the cylinder is identified, then track within the cylinder, and finally, sector within the track.**
- Thus, before discussing the scheduling algorithm, let us look at the criteria of disk-scheduling algorithms.





DISK-SCHEDULING CRITERIA...

- ❑ **Seek Time:** Seek time is the time taken by the disk head to move from one cylinder to another.
- ❑ It is therefore dependent on the **seek length**, that is, the length between the current and the next position of the head.
- ❑ For example, if the disk head is currently at Cylinder number 13 and it has to move to 34. The seek length in this case is 21, that is, to reach its target, the head needs to pass 21 cylinders.
- ❑ The average seek time can be calculated if the average time to move from one cylinder to another is estimated.





DISK-SCHEDULING CRITERIA...

- Based on the mechanical characteristics of the disk, **the seek time is affected by two factors: mechanical settling time and acceleration factor.**

- The seek time is thus given as

$$\begin{aligned}\text{Seek}(n) &= 0 \text{ if } n = 0 \\ &= a + bn \text{ if } n > 0,\end{aligned}$$

- where **n** is the seek distance, a is the mechanical settling time, and b is the acceleration factor.
- **Thus, seek time may affect the performance of the system. The scheduling algorithm that provides the minimum average seek time is better.**





DISK-SCHEDULING CRITERIA...

- **Rotational Latency:** Rotational latency is the time taken by the addressed sector of the track to rotate into a position such that read/write head is accessible to it, that is, the sector comes under the head. The disk scheduling algorithm that provides minimum rotational latency is better.
- **Transfer Time:** Transfer time is the time for data transfer. For example, if it needs to read some data from the disk, then the transfer time is the time taken to transfer the data from the disk to the application. It depends on the rotational speed of the disk and the number of bytes to be transferred.





DISK-SCHEDULING CRITERIA...

- **Disk Access Time:** It is the time to perform any operation on the disk. It is the sum of seek time, rotational latency, and transfer time.
- **Disk Bandwidth:** It is the total number of bytes transferred divided by the total time between the first request of the service and completion of the last transfer of the data. The scheduling algorithm should provide a high bandwidth.
- **Throughput:** It is the total number of disk requests serviced per unit time. A scheduling algorithm must provide maximum throughput.





DISK-SCHEDULING CRITERIA...

- **Response Time:** It is the average time spent by a request in waiting. A request must be serviced within a predictable response time; otherwise, it may have adverse effects on the application.
- There are two parameters in response time:
- One is the average response time that shows the response time of the whole system, and another is variance of response time.
- **Variance of response time:** It is the measure of how individual requests are serviced relative to the average system performance.





DISK-SCHEDULING CRITERIA...

- If the variance is low, then it means that most of the requests are serviced after waiting for a similar amount of time. In this way, the variance is a parameter for measuring the fairness and predictability. To avoid irregularity in service time of the requests, the scheduling algorithm should minimize the variance of response time. The time parameters of the disk are illustrated in Fig. 15.1.

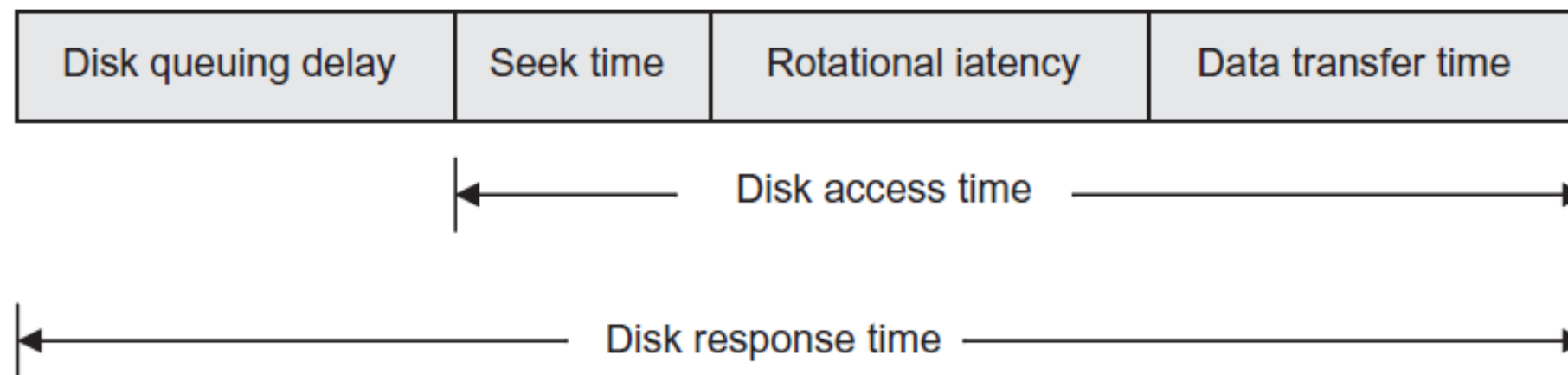


Fig. 15.1 Disk I/O performance time factors





FCFS: DISK-SCHEDULING ALGORITHM

- It is the simplest scheduling algorithm, as seen in case of CPU scheduling.
- The requests are scheduled in the order they arrive in the disk queue.
- Thus, every request gets a **fair chance** to be executed and indefinite postponement of requests does not occur.
- This may, however, not be a good choice for a scheduling algorithm. It may be possible that the first request in the queue is very far from the current position of the disk head. **In this case, the seek time will be high and throughput will be low.**





FCFS: DISK-SCHEDULING ALGORITHM

- ❑ This algorithm may be good in performance when the load on the disk is low and the requests are uniformly distributed.
- ❑ However, as the load on the disk increases, FCFS provides poor performance as it increases the average response time.
- ❑ However, the FCFS will provide low variance of response time due to random seek, that is, the arriving disk requests cannot get ahead of already pending requests.



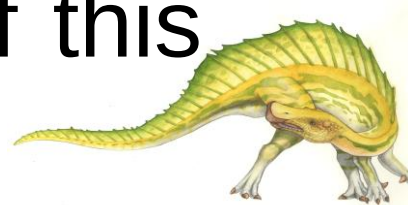


FCFS DISK-SCHEDULING ALGORITHM: Example

- Consider a disk queue with I/O requests on the following cylinders in their arriving order: 54, 97, 73, 128, 15, 44, 110, 34, 45. The disk head is assumed to be at Cylinder 23. Calculate and show with diagram the disk head movement using FCFS-scheduling algorithm.

Solution

- According to FCFS, the head at Position 23 will move to 54, then to 97, and keeps on moving till it reaches the last cylinder as shown in Fig. 15.2. The head movement details are shown in Table 15.1. The head movement is more in this algorithm as it moves from **73 to 128** and **128 to 15** illustrating the inefficiency of this algorithm.





FCFS DISK-SCHEDULING ALGORITHM: Example...

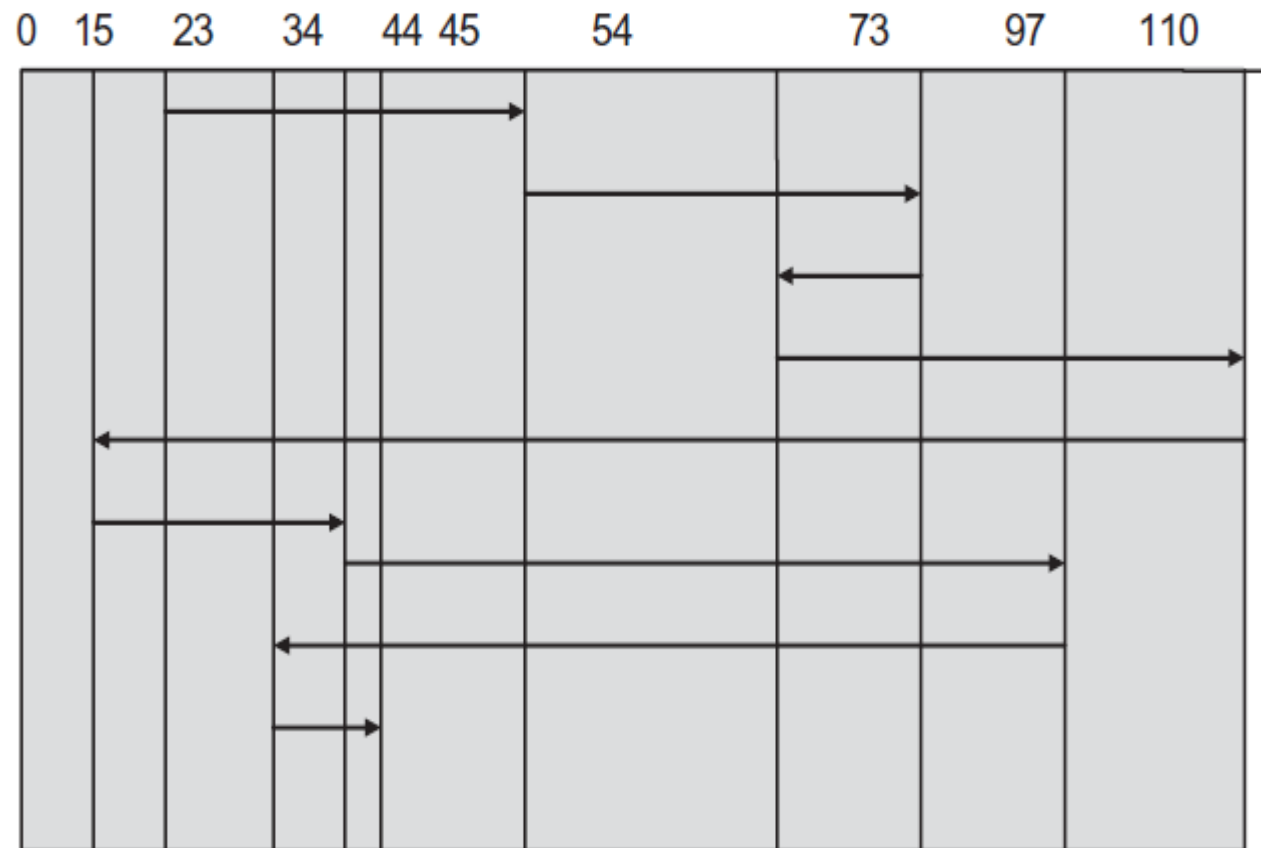


Fig. 15.2 FCFS disk scheduling for Example 15.1

Table 15.1 FCFS head movement for Example 15.1

I/O request for cylinder	Head movement	Total head movement for the request
54	23–54	31
97	54–97	43
73	97–73	24
128	73–128	55
15	128–15	113
44	12–44	32
110	44–110	66
34	110–34	76
45	34–45	11
Total head movement = 451		





SSTF: DISK-SCHEDULING ALGORITHM

- In this algorithm, the request with the **shortest seek time will be executed first.**
- Therefore, it calculates the seek time of all the requests in the queue, selects the appropriate request, and schedules it for the execution.
- In this way, the requests that are near to the current head position get serviced first, thereby improving the performance of the system.
- This will increase the throughput of the system and decrease the average response time as compared to FCFS.





SSTF: DISK-SCHEDULING ALGORITHM

- ❑ Since it needs to calculate the seek time from the current disk head position, it will also incur the cost of computation.
- ❑ Another drawback in this algorithm is that a request very far away from the current head position will not get scheduled if the arriving requests have less seek time compared to this request.
- ❑ In this case, the request will starve.
- ❑ Since the algorithm services the requests whose seek time is less, the requests that are concentrated in one region will be served the most and rest will observe starve.





SSTF: DISK-SCHEDULING ALGORITHM

- ❑ This may cause in low response times for the requests whose seek time is high, that is, the track for the request is at the other end of the disk.
- ❑ Further, it may be difficult to predict the response time as the algorithm favours only some requests. Thus, it causes high variance of the response time.
- ❑ The SSTF algorithm may be best suitable for batch-processing systems where high throughput and average response time are required.
- ❑ However, it may not be efficient for interactive and online systems where high response time and its predictability are required.





SSTF: DISK-SCHEDULING ALGORITHM: Example

- Calculate and show with diagram the disk-head movement using SSTF-scheduling algorithm as Example shown in ppt 31.
- Since in SSTF, the head movement is based on the shortest seek time, calculate the shortest distance that a head needs to move.
- From the current position, the next request whose movement distance is the shortest is serviced, as shown in Table 15.2 and Fig. 15.3.





SSTF: DISK-SCHEDULING ALGORITHM: Example

Table 15.2 SSTF head movement for Example 15.2

Head movement	Total head movement for the request
23–15	8
15–34	19
34–44	10
44–45	1
45–54	9
54–73	19
73–97	24
97–110	13
110–128	18
Total head movement = 121	

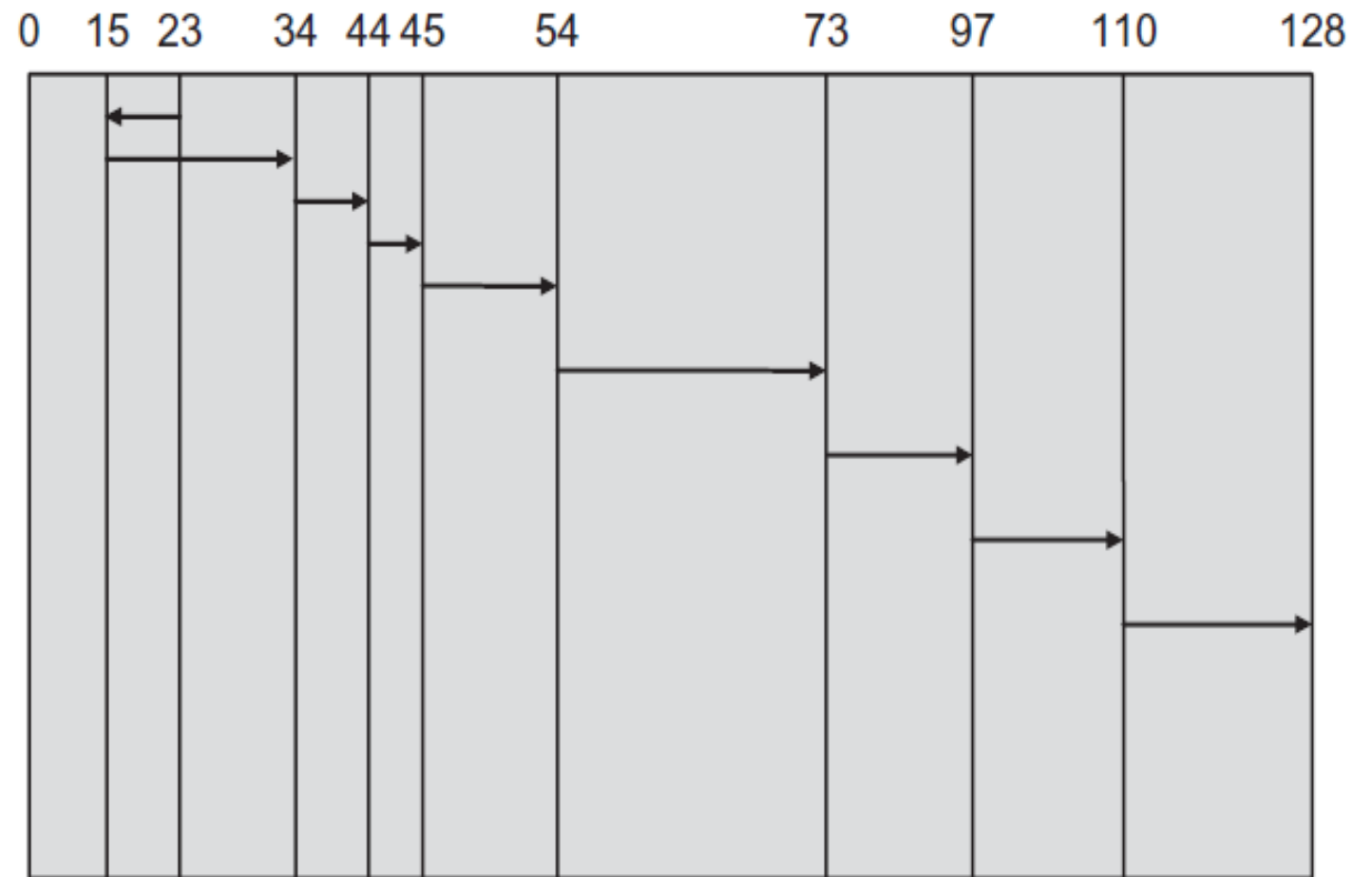


Fig. 15.3 SSTF disk scheduling for Example 15.2

The total head movement drastically reduces in the SSTF algorithm compared to FCFS. Moreover, the head does not change its direction of movement frequently as was seen in case of FCFS. This shows that this algorithm increases the performance of the system as the head movement is reduced by following the shortest seek length.





SSTF: DISK-SCHEDULING ALGORITHM: Example

- In Example shown on previous ppt, suppose when the head is processing request at Cylinder 73, a request at Cylinder 75 arrives. Since the new request will have the shortest seek length, **the head will move to it and service the request.** After this, the head will move to Cylinder 97 and processes the request. While processing, two new requests arrive at Cylinders 99 and 100. Again, these new requests have the shortest seek lengths, the head will move to Cylinder 99 and then to 100.
- **This will make some requests in the queue such as 110 and 128 to starve if more requests whose seek lengths are short continue to arrive in the system.**





SCAN: DISK-SCHEDULING ALGORITHM

- ❑ SCAN uses the property that the disk arm after reaching the end of the disk moving in one direction, that is, the outermost or the innermost cylinder, reverses its direction.
- ❑ It services the requests in the direction of its head movement and continues until it reaches the end of the disk or the last cylinder. After this, it reverses the direction and services the requests in its path.
- ❑ According to this algorithm, the preferred direction of the head movement has to be found first and all the requests in that direction will be serviced.





SCAN: DISK-SCHEDULING ALGORITHM...

- It may be possible that some requests appear while processing a request. If these new requests are in the path of the preferred direction of movable disk head, then these requests will also be serviced. **However, if they arrive just behind the head, then they will have to wait until the head reverses its direction and services them.** In this way, the head moves back and forth across the disk and services the requests in its path.
- Hence, the algorithm is also known as **elevator algorithm** taking the **analogy of an elevator** that continues in one direction servicing each floor as per the requests and reverses its direction after reaching the top or ground floor.





SCAN: DISK-SCHEDULING ALGORITHM...

- Since the behaviour of this algorithm is similar to SSTF, it also provides high throughput and average response time.
- Moreover, it will provide low variance of response time as compared to SSTF as it services all requests in a given direction.
- In this algorithm, the requests at the midrange will be visited more as compared to outer or inner tracks.
- Moreover, if the new requests are arriving in front of the head, then they will be serviced and the other pending old requests (behind the head) will have to wait, causing starvation of some requests.





SCAN: DISK-SCHEDULING ALGORITHM-Example

- Consider a disk queue with I/O requests on the following cylinders in their arriving order: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45. The disk head is assumed to be at Cylinder 23 and moving in the direction of decreasing number of cylinders. The disk consists of total 150 cylinders. Calculate and show with diagram the disk head movement using SCAN-scheduling algorithm.





SCAN: DISK-SCHEDULING ALGORITHM-Example...

Solution

- Since the disk head is at Position 23 and moving in the direction of **decreasing number of cylinders**, it will first service the requests 15, 12, 10, and 6 and move towards the end of the disk, that is, **Cylinder 0**. Then it reverses its direction and services other requests in its path till it reaches Cylinder 150 as shown in Fig. 15.4. The total head movement in this algorithm is **$23 + 150 = 173$** .
- However, in this algorithm, the disk movements from Cylinders 6 to 0 and from Cylinders 128 to 150 are unnecessary as there are no requests in these paths.





SCAN: DISK-SCHEDULING ALGORITHM-Example...

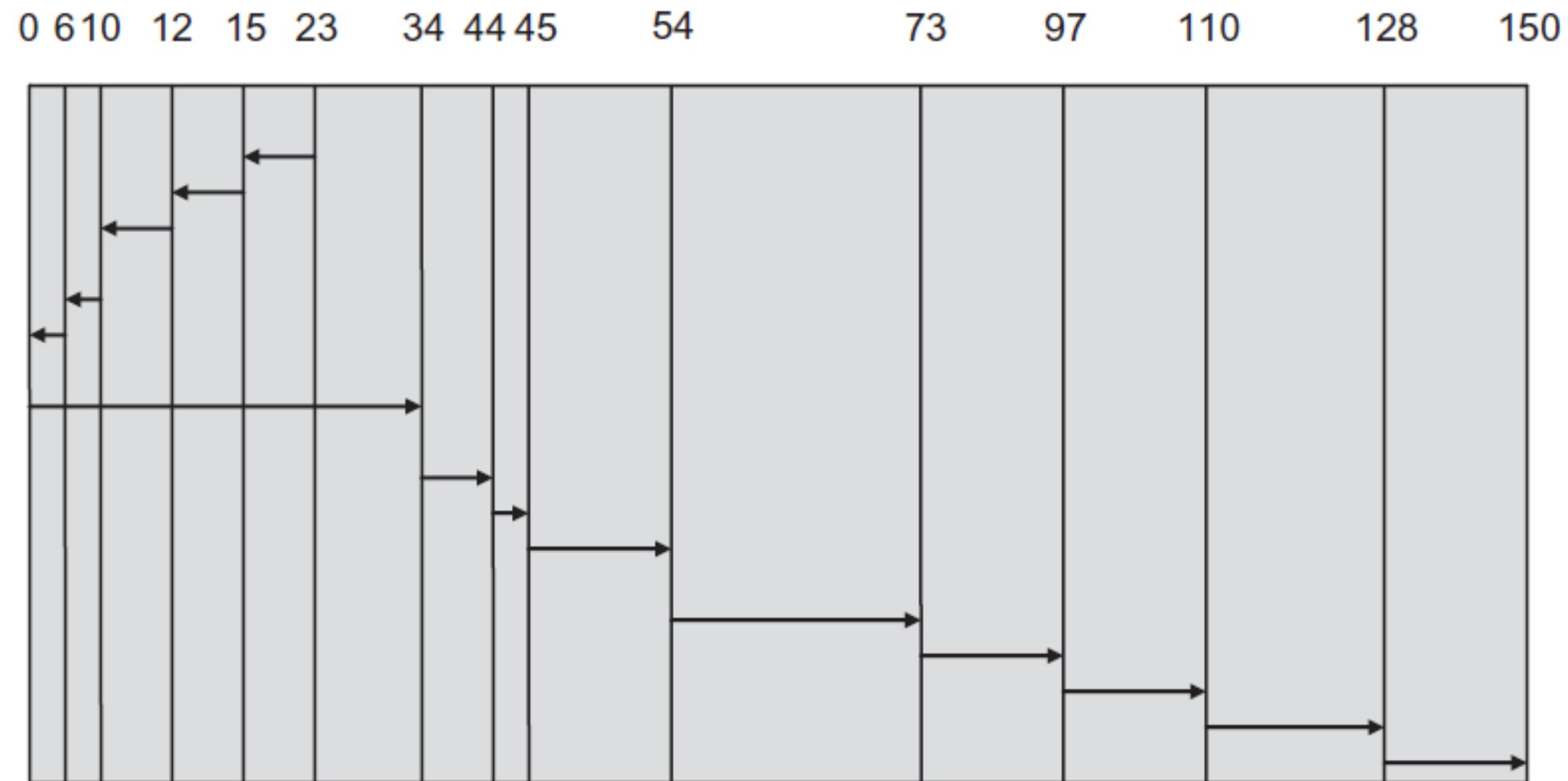


Fig. 15.4 SCAN disk scheduling for Example 15.4





SCAN: DISK-SCHEDULING ALGORITHM-Example...

In Example 15.4, suppose some new requests arrive at Cylinders 60, 65, and 70 while the disk head is processing Cylinder 54. What will happen to these new requests according to the SCAN-scheduling algorithm?

Solution

Since the new requests arrived are in the preferred direction of disk-head movement, all these requests will be serviced just after the processing at Cylinder 54 as shown in the highlighted portion in Fig. 15.5. However, due to arrival of these new requests, the waiting time of pending requests in the queue will increase.





SCAN: DISK-SCHEDULING ALGORITHM-Example...

0 6 10 12 15 23 34 44 45 54 60 65 70 73 97 110 128 150

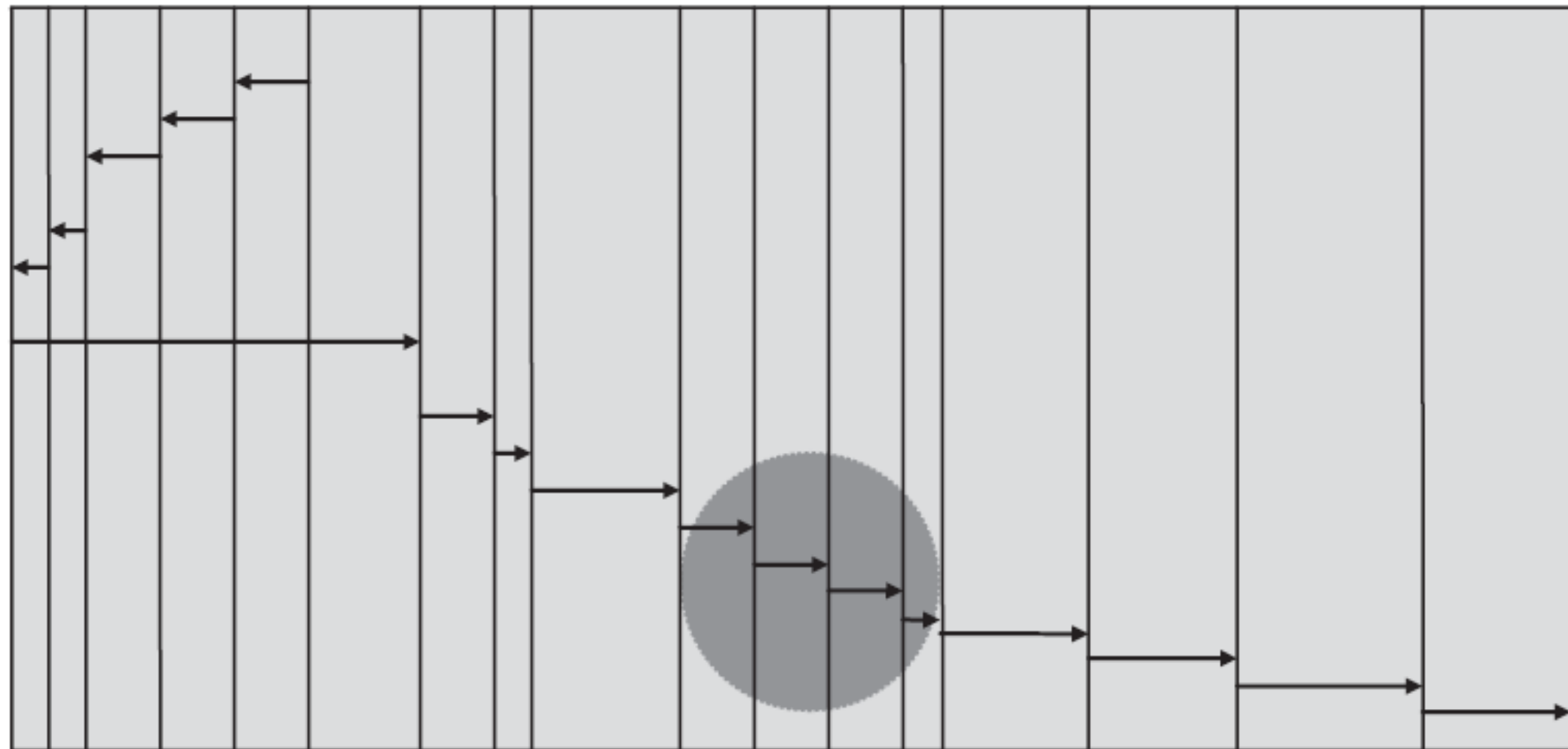


Fig. 15.5 SCAN disk scheduling for Example 15.5



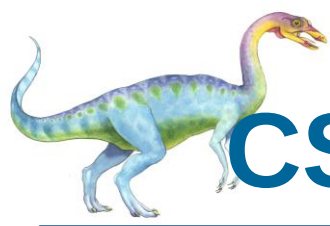


CSCAN: DISK-SCHEDULING ALGORITHM

- In SCAN, the head reverses its direction when it reaches the end of the disk. After reversing its direction, it scans the path for the requests that have already been scanned and serviced.
- Two situations that may arise in case are as follows:
 - Zero or very few requests pending.
 - Many requests waiting on the other end of the disk.

This happens when more and more requests arrive just behind the head while processing. These drawbacks can be overcome by using C-SCAN algorithm. In this, the head, instead of reversing its direction, returns to the beginning of the disk without serving any request in its path and scans and services the requests again in the earlier direction.





CSCAN: DISK-SCHEDULING ALGORITHM...

- In this way, this algorithm considers the line of cylinders as a circular queue such that it scans them repeatedly in one direction.
- This is the reason this algorithm is known as C-SCAN-scheduling algorithm. Since C-SCAN is a modification SCAN algorithm, it maintains its basic nature of providing high throughput. Further, it provides the low variance in response time as compared to SCAN algorithm as the pending jobs on the other side of the disk may get chance to be executed and centre cylinders are not favoured. However, the starvation may still occur as in SCAN algorithm if more and more new requests arrive on the same cylinder or in front of the head.

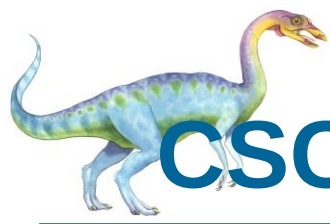




CSCAN: DISK-SCHEDULING ALGORITHM: Example

- **Solve Example shown on PPt 41** using C-SCAN-scheduling algorithm.
- **Solution**
- The disk head after reaching Cylinder 0 reverses its direction, and starts scanning from Cylinder 150 and services the requests in its path as shown in Fig. 15.6. The total head movement in this algorithm will be reduced to 139 as compared to 173 of SCAN algorithm.





CSCAN: DISK-SCHEDULING ALGORITHM: Example

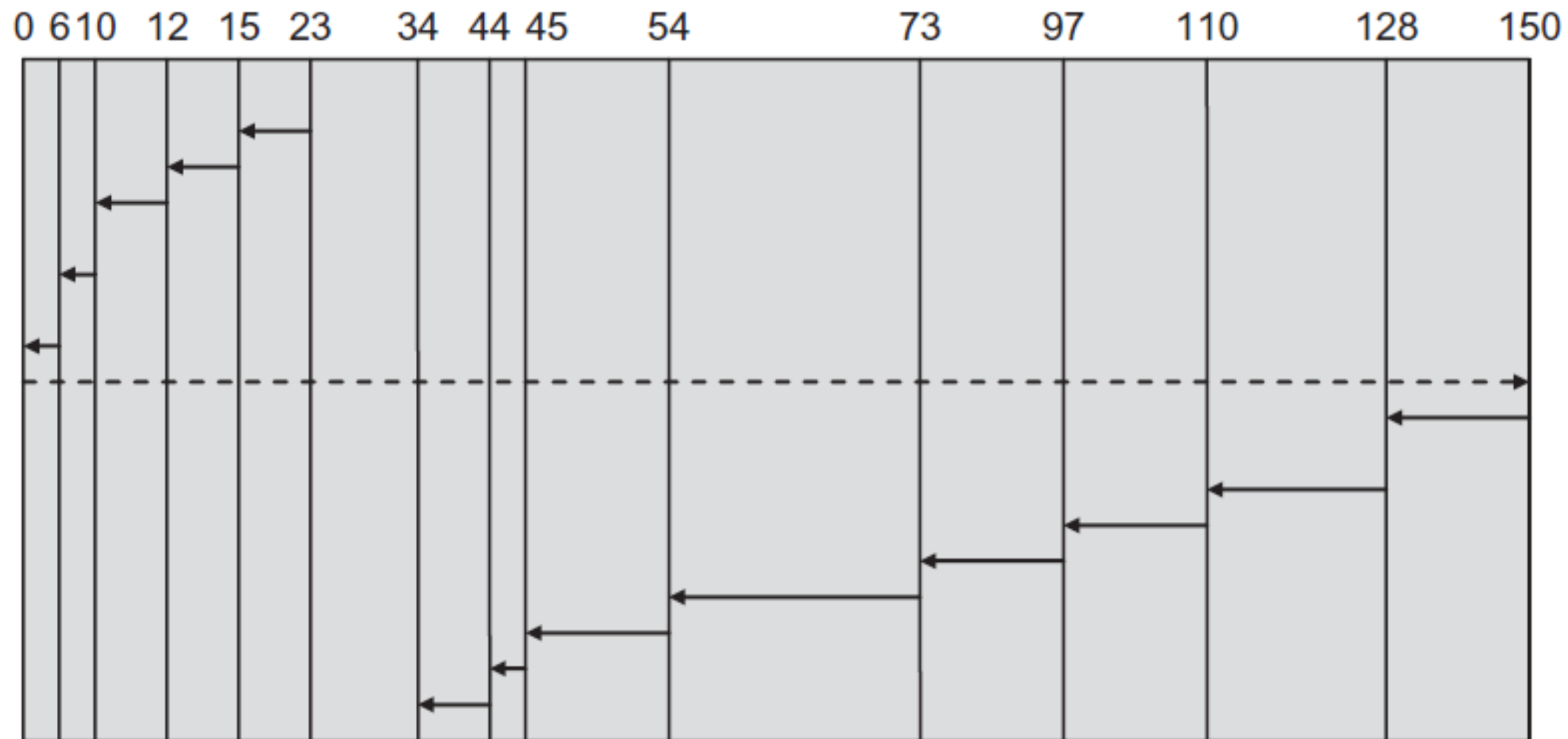


Fig. 15.6 C-SCAN disk scheduling for Example 15.6





F-SCAN and N-step SCAN: DISK-SCHEDULING ALGORITHM

- ❑ In all the algorithms discussed earlier, if new I/O requests arrive while the disk head is processing one request, then some requests may starve and are denied fair chance to be serviced.
- ❑ To resolve this problem, some variants of scan algorithm have been developed:
- ❑ F-SCAN and N-step SCAN.

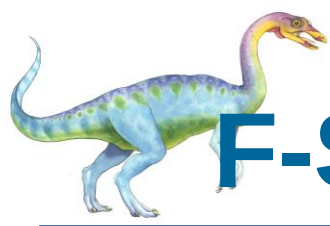




F-SCAN: DISK-SCHEDULING ALGORITHM

- ❑ It maintains two queues. One queue is for storing the requests that have arrived and another queue is initially empty. The disk arm starts servicing the requests from the first queue.
- ❑ However, if there are any new requests arriving while old requests from the first queue are being processed, then they will reside in the second queue; they will not be allowed to enter the first queue and thereby change the order of the queue.
- ❑ In this way, the new requests are deferred until all the pending old requests have been processed.





F-SCAN: DISK-SCHEDULING ALGORITHM...

- However, this freezing of requests is for the period till the disk head reaches the end of the disk.
- As the disk head reaches the last cylinder and changes its direction, the second queue is merged with the first queue and sorted for optimum service.
- The disk head now starts servicing the requests according to this new queue of the requests.





F-SCAN: DISK-SCHEDULING ALGORITHM-Example

- ❑ Consider a disk queue with I/O requests on the following cylinders in their arriving order: 54, 97, 73, 128, 15, 44, 110, 34, 45.
- ❑ The disk head is assumed to be at Cylinder 23 and moving in the direction of decreasing number of cylinders. The disk consists of total 150 cylinders.
- ❑ Suppose when the head is processing at 15, a new request arrives at Cylinder 10. Similarly, when the head is processing request at 73, a request on Cylinder 75 arrives. While processing at 97, new requests arrive at Cylinders 35, 99, and 100. Apply the F-SCAN algorithm for this situation.





F-SCAN: DISK-SCHEDULING ALGORITHM-Example...

- Using F-SCAN, two queues are maintained as shown in Fig. 15.7.

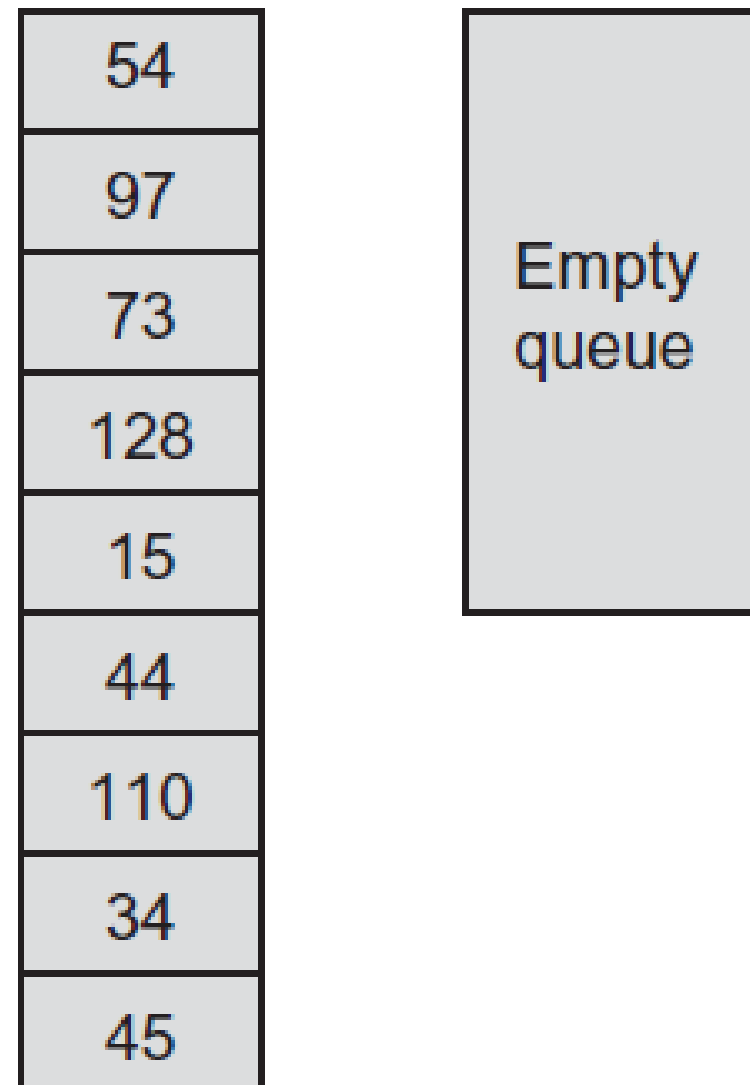


Fig. 15.7 F-SCAN initial queues for Example 15.7





F-SCAN: DISK-SCHEDULING ALGORITHM-Example...

- When the head processes at **15**, a request arrives at Cylinder **10**. However, this will not be processed as the original queue is frozen and hence, it is added to the second queue. As the head reaches the end of the disk, that is, at Cylinder 0, it reverses its direction. At this point, the original queue is merged with the second queue and sorted in the ascending order as shown in Fig. 15.8. **This sorted queue is now the original queue.**

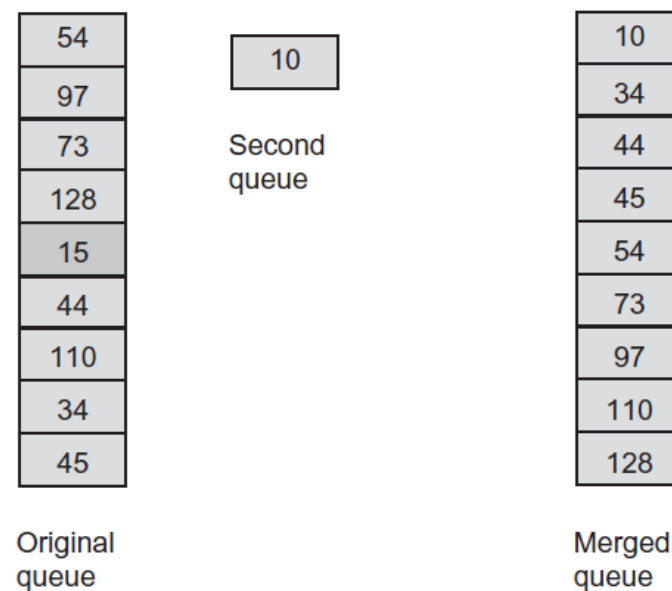


Fig. 15.8 F-SCAN first pass for Example 15.7





F-SCAN: DISK-SCHEDULING ALGORITHM-Example...

- When the head processes at 73, a request arrives at Cylinder 75. The request is added in the second queue. Further, while processing at 97, new requests arrive at Cylinders 35, 99, and 100. The requests are again added in the second queue. The new status is shown in Fig. 15.9.

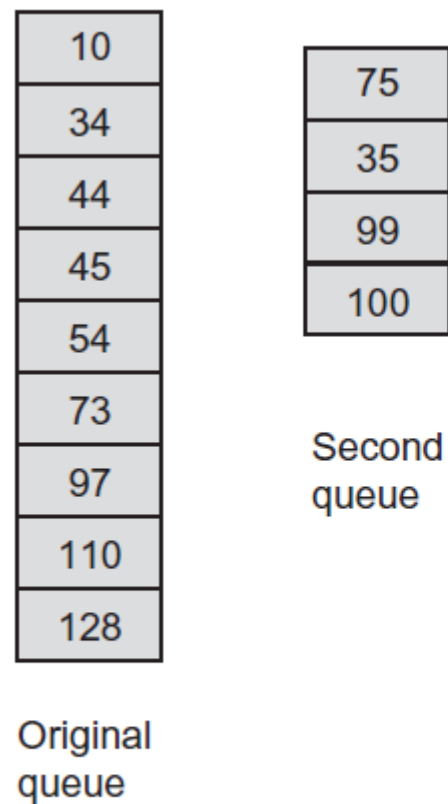


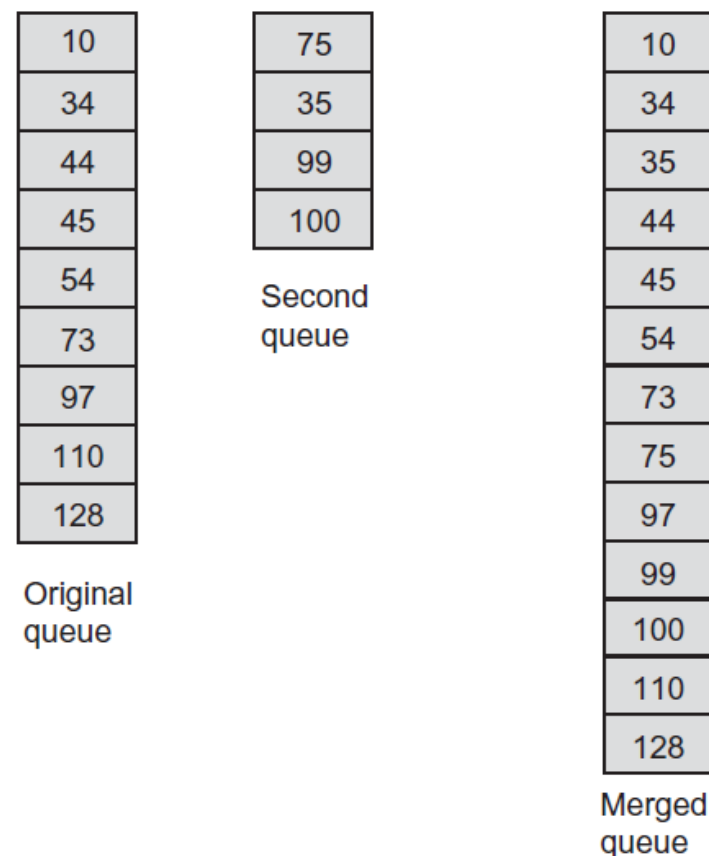
Fig. 15.9 F-SCAN status of queues after first pass for Example 15.7





F-SCAN: DISK-SCHEDULING ALGORITHM-Example...

- However, no new requests present in the second queue will be processed as the head has not reached the end of the disk. **As soon as the head reverses its direction, the original list is merged with the second queue and sorted and processed again.** The statuses of the queues are shown in Fig. 15.10.





F-SCAN: DISK-SCHEDULING ALGORITHM-Example...

- Since in our example, all the requests are serviced in the original queue, the second queue is the remaining queue that needs service. This queue is sorted and then serviced as shown in Fig. 15.11.

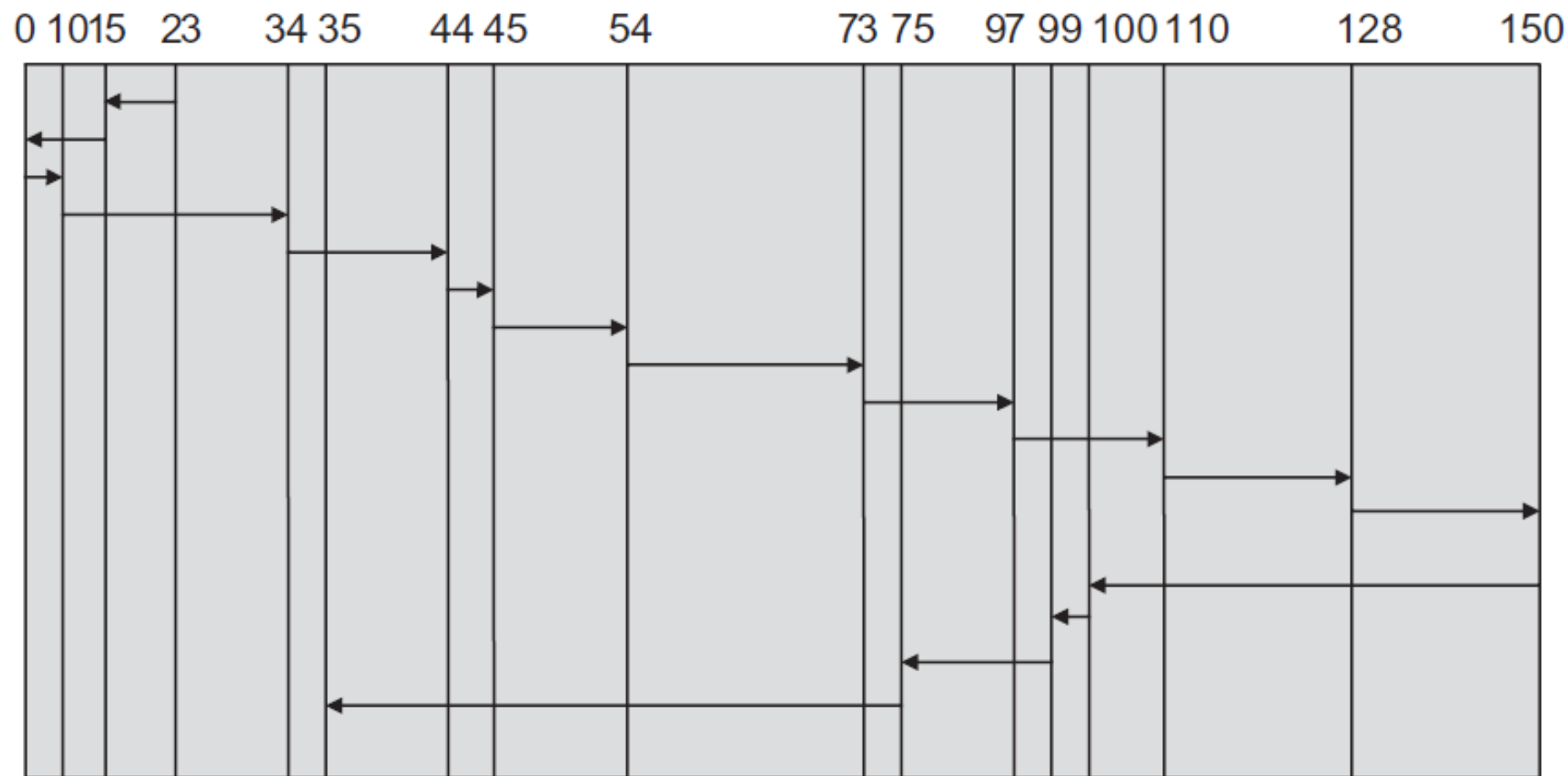


Fig. 15.11 F-SCAN disk scheduling for Example 15.7





N-Step-SCAN: DISK-SCHEDULING ALGORITHM

- The idea here is to serve the requests at the interval of n in the queue in a sweep when the head is moving in one direction. For example, if n is four, the first four requests are serviced first. When the sweep is complete, the next four requests in the queue are considered for service. In this way, a large queue is divided into sub-queues, each containing n requests. When a new request appears while a sub-queue is being processed, it will be added at the end of the original queue. This helps to avoid starvation, thereby guaranteeing the service of the requests. The value of n should be chosen carefully for the maximum performance of the algorithm. If value of $n = 1$, then it becomes a first-in-first-out (FIFO) algorithm.





N-Step-SCAN: DISK-SCHEDULING ALGORITHM...

- On the other hand, if the value of n is very large, then it is as good as SCAN algorithm. Neither of the variants of SCAN algorithms therefore postpones the service of any old pending requests and thereby reduces the variance of the response times as compared to SCAN algorithm. The F-SCAN may prove to be better as compared to N-SCAN algorithm.
- N-SCAN algorithm may decrease the throughput as it adds the new arriving requests at the end of request queue and these cannot be serviced until all sub-queues have been serviced. However, F-SCAN algorithm considers the new requests in its original queue after every sweep when the disk head changes its preferred direction.





N-Step-SCAN: DISK-SCHEDULING ALGORITHM...

□ The following are the benefits of F-SCAN over N-SCAN:

- High throughput
- Low variance of the response time
- Good average response time
- Reduced starvation





N-Step-SCAN: DISK-SCHEDULING ALGORITHM Example

- Consider a disk queue with I/O requests on the following cylinders in their arriving order: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45. The disk head is assumed to be at Cylinder 23 and moving in the direction of decreasing number of cylinders.
- The disk consists of total 150 cylinders.
- Suppose when the head is at 15, a request arrives at Cylinder 11 and when it is at 73, a request arrives at Cylinder 75. While at 97, new requests arrive at Cylinders 35, 99, and 100. Apply the N-SCAN algorithm for this situation for $n = 4$.





N-Step-SCAN: DISK-SCHEDULING ALGORITHM Example...

Solution

- Since the $n = 4$, the first four requests in the queue, that is, **6, 10, 12, and 54** are considered for processing. Now, the head is presently at 23 and moving towards 0. Therefore, it processes the requests **12, 10, and 6 in its scan towards 0 and then reverses its direction for the next sweep.** The request 54 though is in the consideration but could not be serviced as it was not found in its previous sweep.
- Therefore, it is considered in the next sub-queue. The next four elements in the sub-queue, **54, 97, 73, and 128**, are serviced.





N-Step-SCAN: DISK-SCHEDULING ALGORITHM Example...

When the head was processing request at 73, a request for Cylinder 75 arrived. Although it was in the path of the last sweep, it could not be serviced as the algorithm does not allow and was added at the end of the queue. Similarly, while processing at 97, new requests arrived for Cylinders 35, 99, and 100 but could not be processed. These are also added at the end of the original queue. The head again reverses its direction. The next sub-queue having the elements **15, 44, 110, and 34** is serviced. When the head was servicing at 15, a new request for Cylinder 11 appears. However, it was not considered for service and was added at the end of the original queue. In this way, the next subqueue consisting of **45, 75, 35, and 99** is serviced.





N-Step-SCAN: DISK-SCHEDULING ALGORITHM Example...

Finally, we have only two requests, 100 and 11, in the queue, which are serviced as shown in Fig. 15.12.

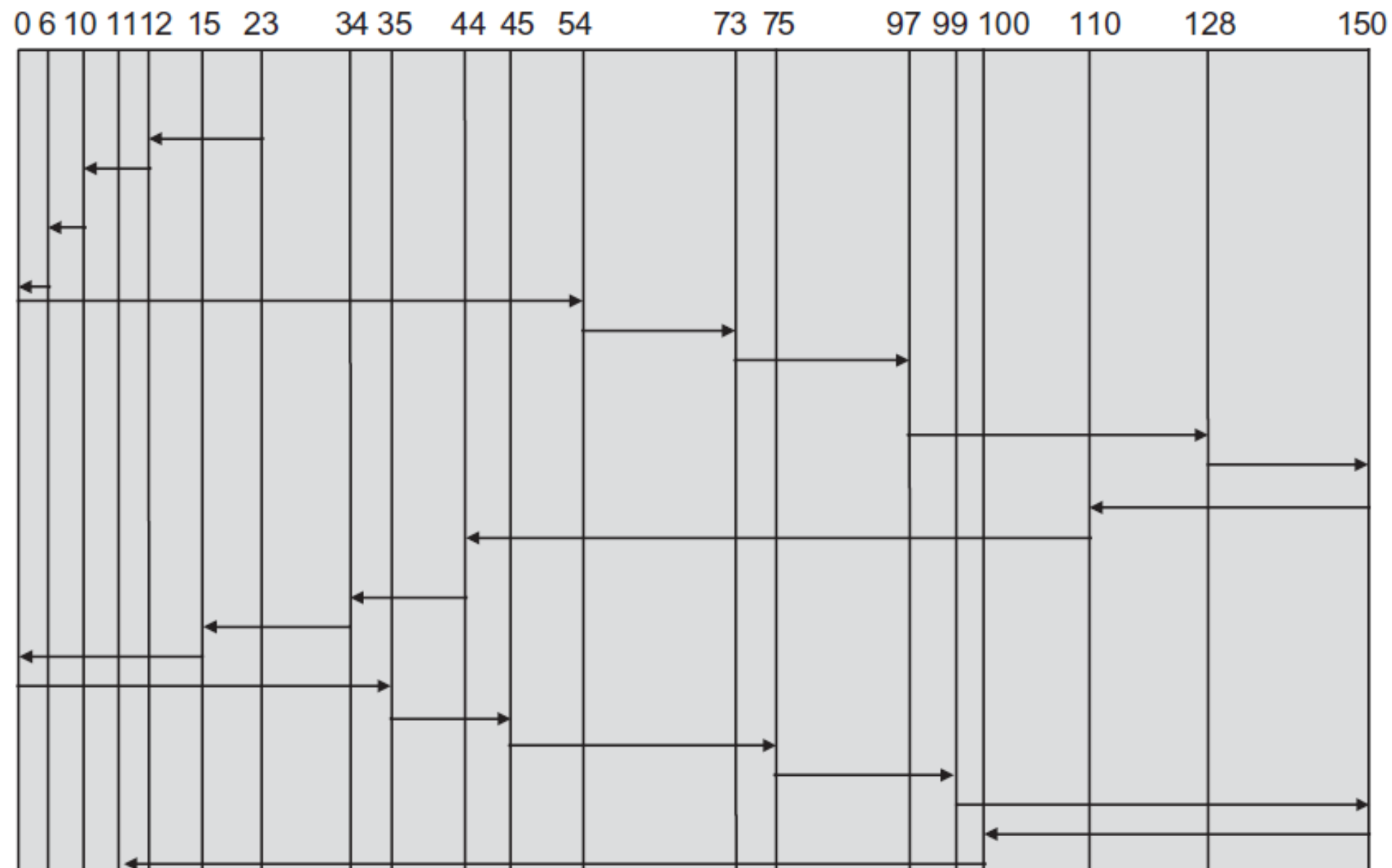
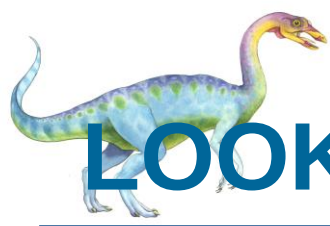


Fig. 15.12 N-SCAN disk scheduling for Example 15.7

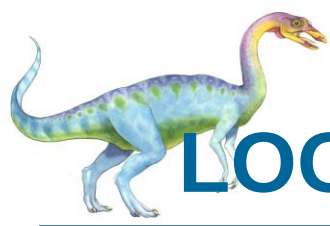




LOOK and C-LOOK: DISK-SCHEDULING ALGORITHMS

- ❑ In SCAN and its variants, the disk head moves to the last cylinder and then reverses its direction. In this case, the head moves unnecessarily, thereby increasing the delay. Therefore, another modification is made in these algorithms such that the head moves in one direction only till the last request in that direction.
- ❑ LOOK algorithm solves this deficiency. According to this, the disk head looks ahead in its direction of movement for the last request. It moves to the last request and changes its preferred direction there without moving to the last cylinder and continues servicing the request in the other direction. Thus, it eliminates unnecessary seek operations and hence decreases the average response time and lowers the variance of response time.

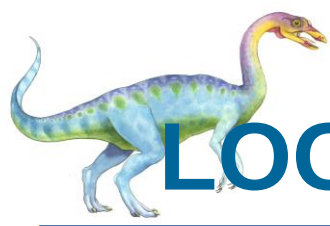




LOOK and C-LOOK: DISK-SCHEDULING ALGORITHMS...

- Like SCAN algorithm, LOOK can also be designed as C-LOOK that treats the request queue as circular.
- It is similar to C-SCAN except that the head moves till the last request instead of last cylinder. Thus, C-LOOK achieves the benefits of both C-SCAN- and LOOK-scheduling algorithms.
- C-LOOK lowers the variance of the response time as compared to LOOK algorithm but incurs cost of reduced throughput. It may increase the average response time also.

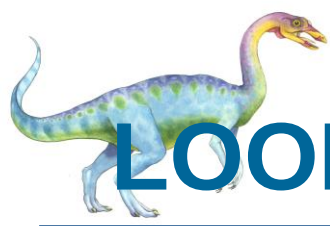




LOOK : DISK-SCHEDULING ALGORITHMS Example

- Consider a disk queue with I/O requests on the following cylinders in their arriving order: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45
- The disk head is assumed to be at Cylinder 23 and moving in the direction of decreasing number of cylinders. The disk consists of total 150 cylinders. Calculate and show with diagram the disk head movement using LOOK-scheduling algorithm.





LOOK : DISK-SCHEDULING ALGORITHMS Example...

Solution

- Since the disk head is at Position 23 and moving in the direction of decreasing number of cylinders, it will first service the requests at 15, 12, 10, and 6. It reverses its direction at 6 and services other requests in its path as shown in Fig. 15.13.

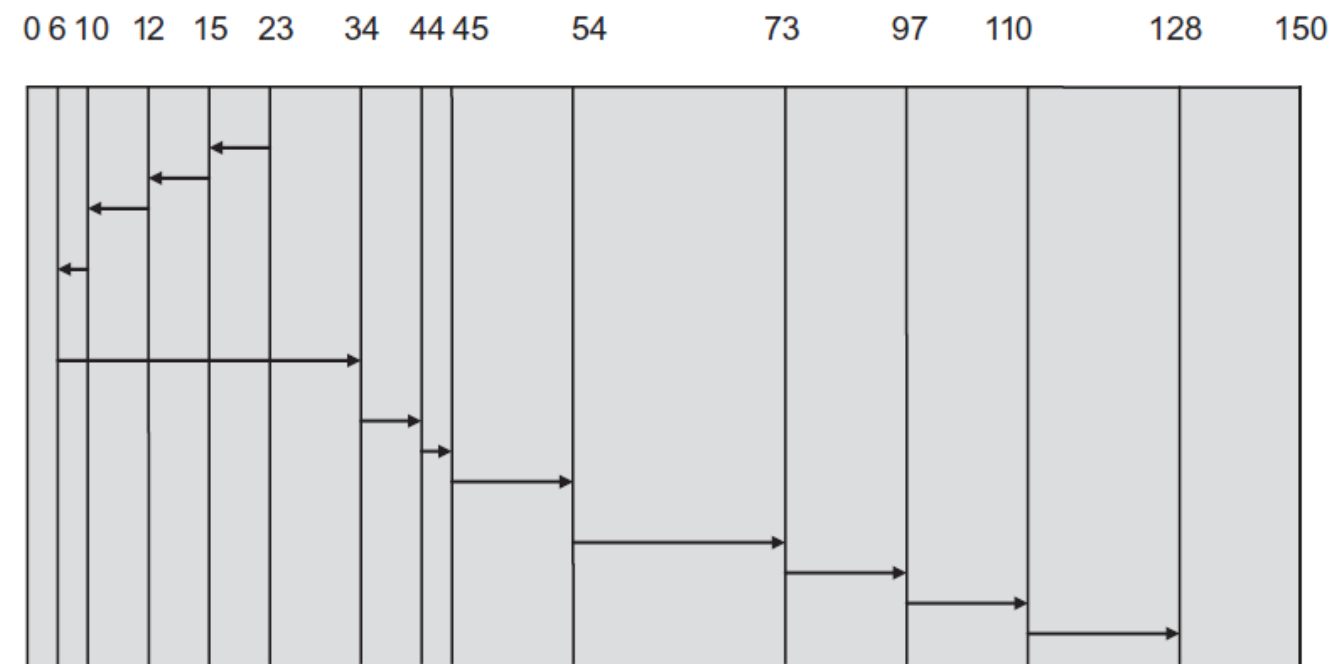
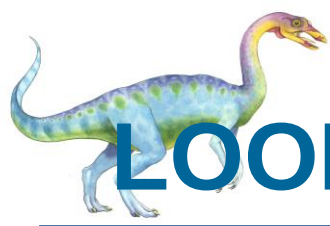


Fig. 15.13 LOOK disk scheduling for Example 15.9

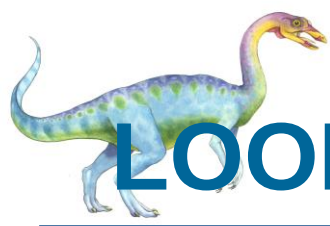




LOOK : DISK-SCHEDULING ALGORITHMS Example...

The head stops at 128 without continuing to the last cylinder and reverses its direction, if there are any new requests pending. The total head movement in this algorithm is 139.

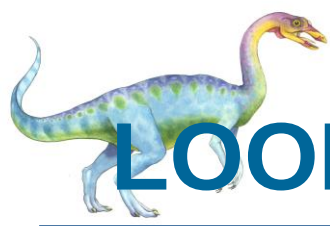




LOOK : DISK-SCHEDULING ALGORITHMS Example...

- Consider a disk queue with I/O requests on the following cylinders in their arriving order: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45
- The disk head is assumed to be at Cylinder 23 and moving in the direction of decreasing number of cylinders. The disk consists of total 150 cylinders. Apply C-LOOK-scheduling algorithm.



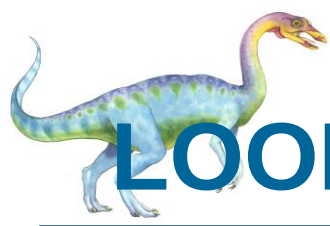


LOOK : DISK-SCHEDULING ALGORITHMS Example...

□ Solution

- The head will first service the requests 15, 12, 10, and 6 and changes its direction. Then it moves to the other end of the disk but looks for the first request there, that is, 128. At Cylinder 128, it reverses its direction and services the requests in its path as shown in Fig. 15.14. The total head movement in this algorithm will be reduced to 111 as compared to 139 of LOOK algorithm.





LOOK : DISK-SCHEDULING ALGORITHMS Example...

0 6 10 12 15 23 34 44 45 54 73 97 110 128 150

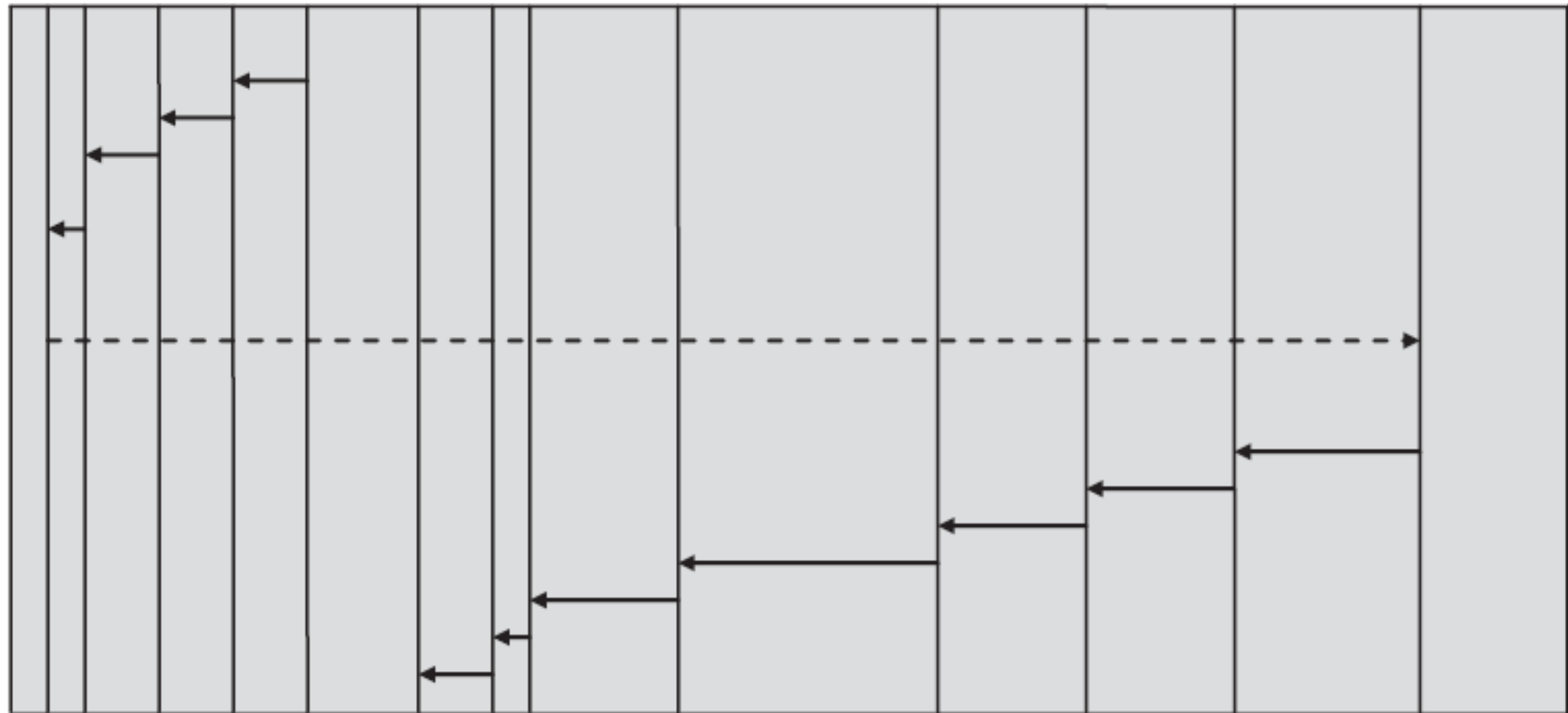


Fig. 15.14 C-LOOK disk scheduling for Example 15.10





S-LOOK : DISK-SCHEDULING ALGORITHM

- Another variant of LOOK algorithm is S-LOOK. The shortest-LOOK (S-LOOK) algorithm is an extension of the LOOK algorithm to handle the cases where the disk head is located among the far-end requests. The algorithm decides which direction should be served first instead of only continuing to seek in the same direction before the new requests have arrived. It calculates the total seek distance for the requests on its both sides. After calculating the seek distances in both directions, it continues to service the requests in the direction where the total seek distance is minimum.
- Thus, S-LOOK provides better average seek time than LOOK algorithm.

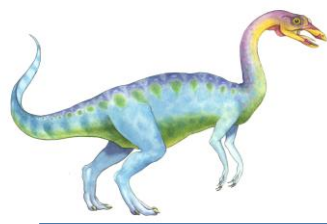




RAID STRUCTURE

- Although the disk is widely used for storage purposes, its slow access as compared to the processor and memory is a critical factor in overall computer system performance. **There has been a performance gain in the disk access in the last few years, but this is not sufficient compared to the access speed of processor and memory.**
- To bridge this gap, an idea was coined by a group of researchers at the University of California; they considered the concept of parallel processing in disk management. Patterson et al. suggested the use of an array of disks instead of one that could be accessed simultaneously to improve the speed as well as reliability.





RAID STRUCTURE...

- These multiple disks may operate independently and in parallel. With multiple disks, it was possible to organize the data in various ways and redundancy of data helps in increasing the reliability.
- The researchers named this new I/O as RAID. The idea of this multiple disk database design was accepted by the industries but changed the 'inexpensive' to 'independent'. This structure then became popular as RAID.
- The RAID structure is a set of physical disk drives, and RAID controller (instead of a disk controller). However, the RAID structure is a single logical drive for the OS. The RAID structure is organized in seven levels (zero to six) of disk arrays.





RAID STRUCTURE...

- These levels do not represent any Hierarchical organization but are different design architectures proposed by the researchers. In RAID, the data is distributed among various disks such that the data is accessed simultaneously from multiple drives, thereby having quick access. While designing the RAID structure, the following two aims are set:
- **Increased performance:** Multiple disks can be accessed in parallel. Therefore, a file may be organized in such a way that the data is spread across multiple disks. Therefore, a file stored on RAID structure is accessed from multiple disks simultaneously, thereby improving the traditional disk accesses.





RAID STRUCTURE...

- Suppose, if two requests belong to two different disks, they can be served in parallel. In this way, more number of requests can be serviced as compared to a single-disk system, increasing the throughput of the system. Moreover, the response time in accessing a large data is reduced.
- **Increased reliability:** In some applications such as database design or real-time system design, reliability is a critical requirement in the system. In these types of systems, the data loss cannot be afforded. Here, the design of RAID can utilize the multiple disk drives to improve the reliability by putting redundant copies of the data. Moreover, the use of multiple drives increases the probability of failure. So the RAID addresses the redundancy issue effectively so as to reduce the chance of loss of data.





RAID STRUCTURE...

- Improvement in performance and reliability is achieved via the following:
- **Data striping:** In RAID, the data is stored on separate disks as a strip. A strip may be a fixed size block on the disk. A file is viewed as an array of contiguous strips, that is, the file data is split among multiple disks in the form of strips. The contiguous strips of a file are stored on separate disks at the same location. The set of strips stored on the disks in the array is known as a stripe. In this way, a file is stored in the form of strips across multiple disks along a stripe. Thus the OS views the array of multiple disks as a logical/virtual single disk. The file data can be processed in parallel depending on the length of the stripe, that is, the number of disks participating. If there are three disks in a stripe, the speed of accessing the data may increase by three times as the file data can be accessed simultaneously as shown in Fig. 15.17.





RAID STRUCTURE...

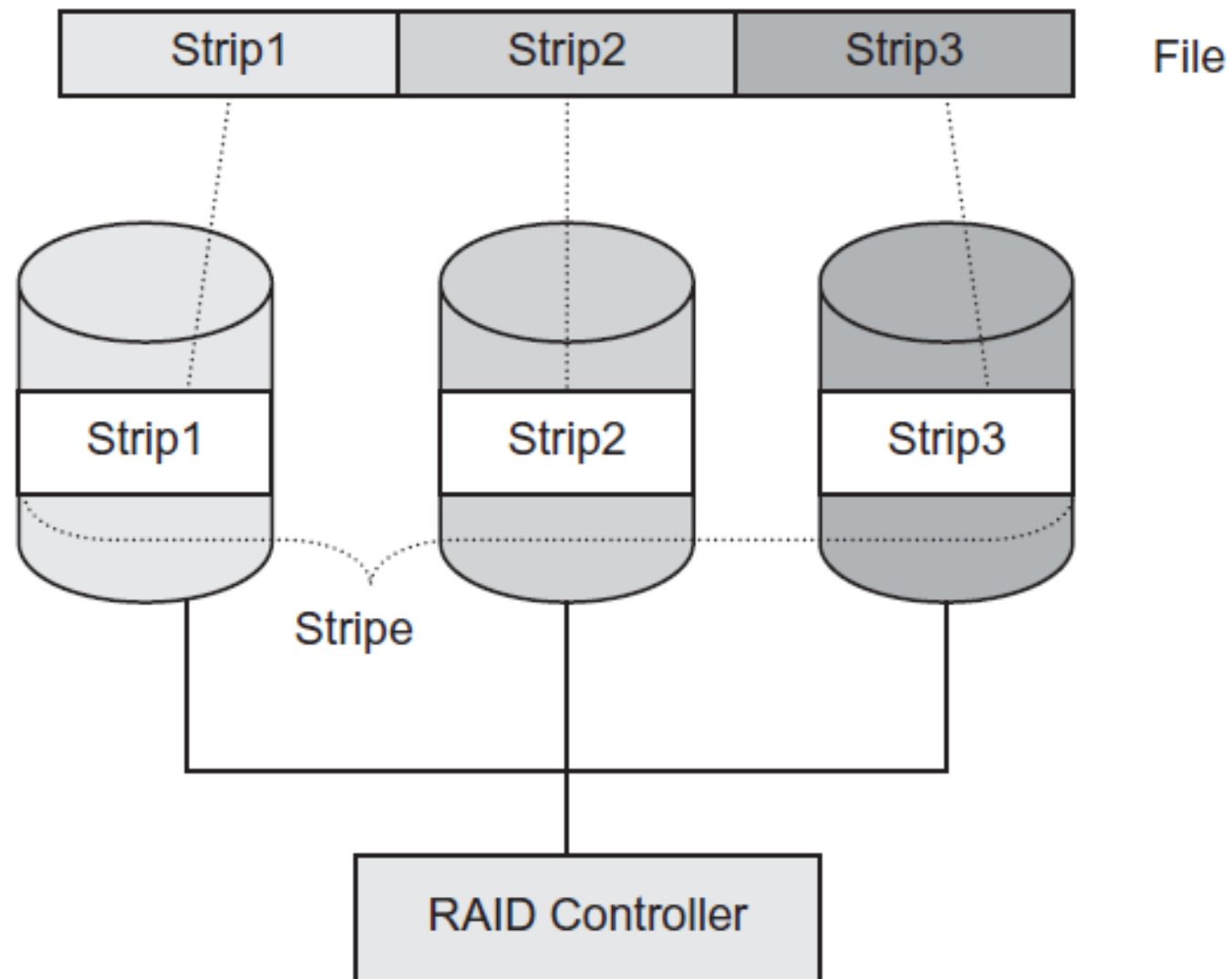


Fig. 15.17 Data striping





RAID STRUCTURE...

- The strip size may affect the access time, throughput, and data transfer time. If the strip size is small, then the number of participating disks in the stripe will be increased.
- The smaller sized strips are known as fine-grained strips. Fine-grained strips will reduce the access time as compared to a single disk and thereby increase the throughput and data transfer rate. If we increase the strip size, then it may be possible that a request will map only one or two strips.
- These large-sized strips known as coarse-grained strips, therefore, service more requests but may reduce the data-transfer rate.





RAID STRUCTURE...

- **Disk mirroring:** As discussed earlier, the RAID structure provides redundancy of data in some systems where the failure of access or loss of data cannot be afforded. The redundancy is provided through the disk-mirroring concept where the duplicate of a strip is placed on a separate disk such that every disk has a mirror copy. If one of the disks fails, then the mirrored copy is accessed. However, mirroring will reduce the storage capacity on RAID structure.
- **RAID controller:** Since the RAID structure consists of multiple disks, a normal disk controller is not able to handle this structure. Instead, RAID controllers are used to manage the disks. They perform data stripping and disk mirroring and maintain redundancy as necessary. For example, if a read request consists of three consecutive strips, then the RAID controller will break this request into three different read commands, mapping them on three disks so that they can be read in parallel.





RAID Levels

- The various RAID levels are discussed as follows:
- RAID Level 0 (Striping): It is the simplest form of RAID structure wherein the data is distributed across all the disks in the form of strips. RAID is like a logical disk that is divided into strips. These strips are mapped into an array of consecutive disks in round robin manner, that is, the first n logical strips are stored as the first strip on each of the disks, and the next n logical strips are stored as the second strip on each disk and so on, forming multiple stripes as shown in Fig. 15.18.

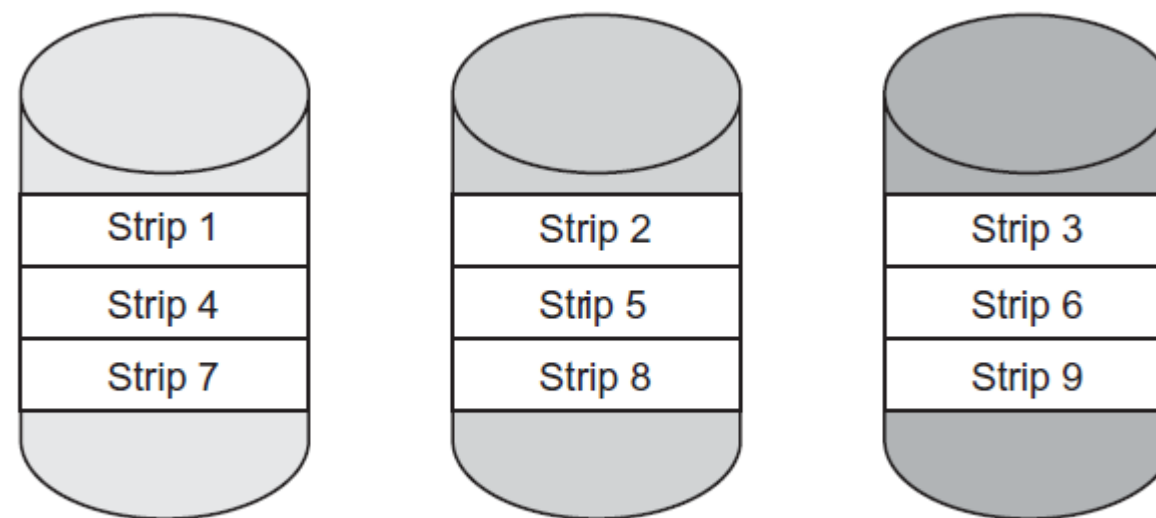


Fig. 15.18 RAID level 0





RAID Levels...

- However, this structure does not provide any redundancy. If one of the disks in the array is not accessible, then the data or strips in that disk are lost.
- Consequently, all the other data retrieved from other disks may become unusable if they are related to a single request or the lost data. Due to this reason, this level is not considered as a true member of RAID structure.
- RAID level 0 is simple to implement as it does not incur any cost for redundancy.
- Moreover, it increases the performance of disk access by the factor n where n is the number of disks in the disk array.
- This structure is more appropriate for the applications that do not require reliability but performance and low cost.





RAID Levels...

- **RAID Level 1 (Mirroring):** This structure includes striping as well as redundancy. The redundancy is achieved via disk mirroring, as shown in Fig. 15.19. When there is a read request, the disk with the shortest seek time and rotational latency can be accessed, thereby having the advantage of distributing the load over more drives. In case there is a write request, every strip needs to be updated. However, this write operation on various strips can be done in parallel. Thus, the efficiency is not affected in write operation.

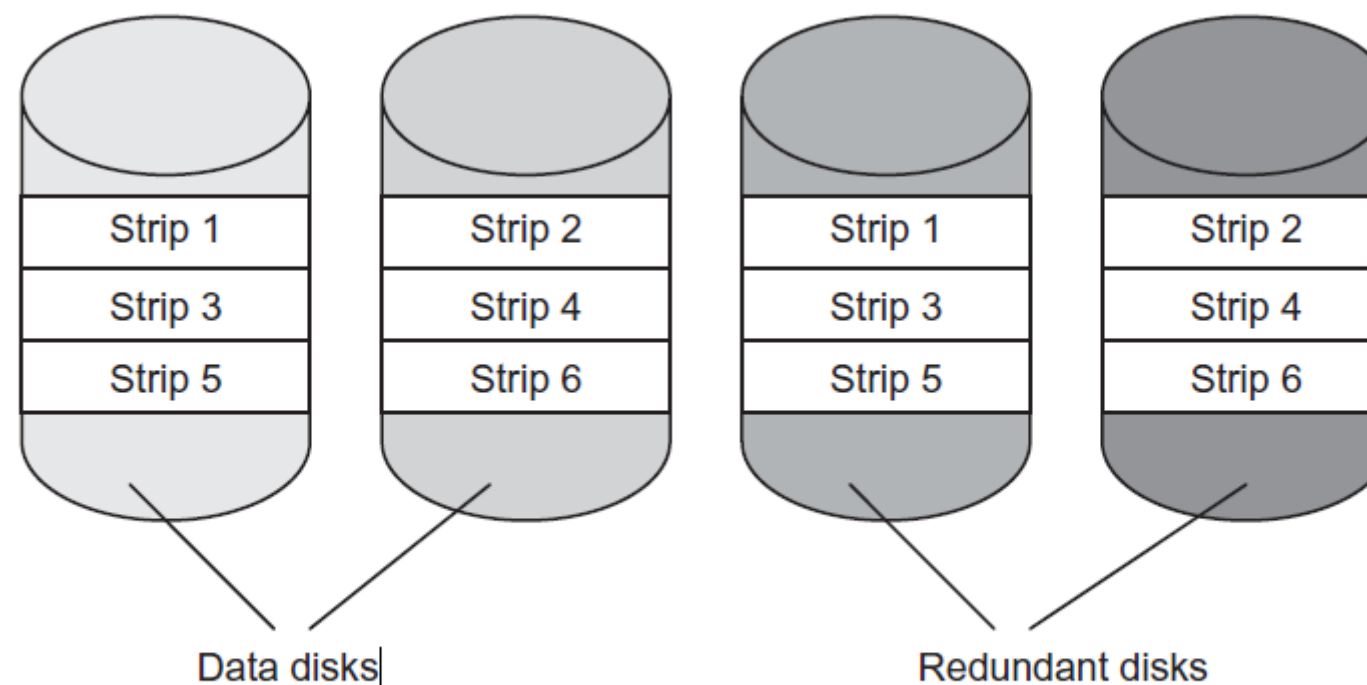


Fig. 15.19 RAID level 1





RAID Levels...

- This structure incurs the cost of having redundant copies of the data on backup disks, thereby reducing the storage capacity. The cost per unit storage, thus, is twice than in RAID level 0.
- Therefore, applications with critical data must use this structure. For these applications, the fault tolerance is quite good as the data may be recovered from the backup disk in case of any disk failure.





RAID Levels...

- ❑ **RAID Level 2 (Fine-grained Striping and ECC Parity):** This structure uses fine-grained strips. Another modification in this structure is that instead of disk mirroring, an ECC scheme is implemented, thereby reducing storage overhead and cost. ECC is used to detect and correct the errors on the data. Hamming ECC is used for this purpose.
- ❑ Hamming code is calculated across corresponding bits on each disk. It uses parity bits to check for errors in data transmission from disks and possibly corrects them. The parity bits of the hamming code are stored in separate disks. For example, there is a data of four bits to be written and each strip stores one bit. In this case, the stripe consisting of four bits is written on four separate data disks where each strip stores one bit. RAID structure then calculates the parity bits corresponding to the data stored in the stripe and then stores them in separate parity disks as shown in Fig. 15.20.





RAID Levels...

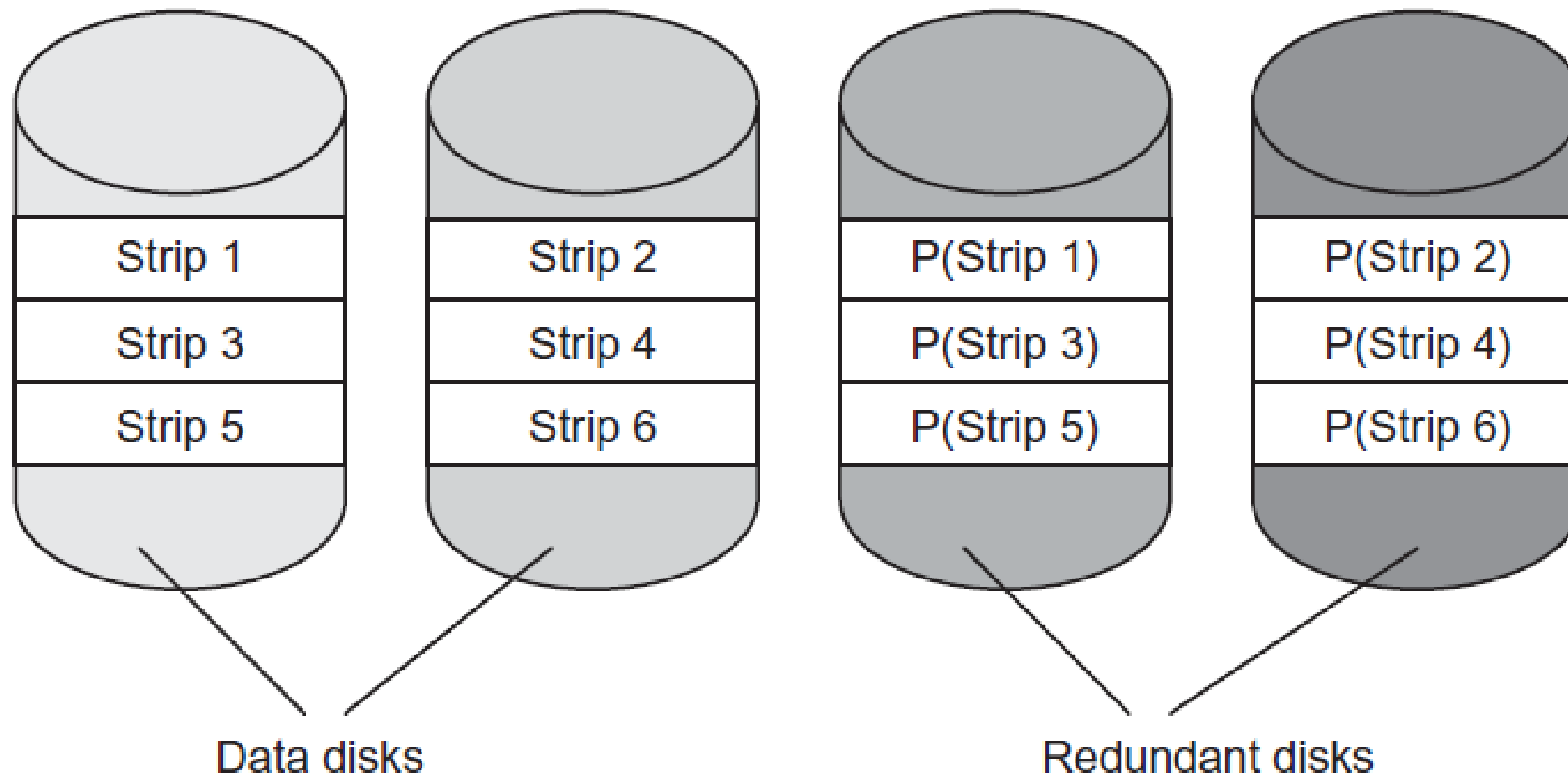


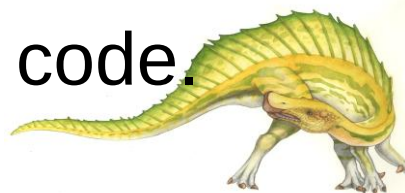
Fig. 15.20 RAID level 2





RAID Levels...

- Thus, this structure reduces data storage, but even then, it is very less as compared to RAID level 1. The increase in number of parity disks is proportional to the log of number of data disks.
- In case of a read operation, all the disks need to be accessed as the request data is spread over all the disks and moreover, the hamming code is calculated and compared with the values retrieved from the parity disks. Similarly, in case of a write operation even on a single strip, all the disks including data and the parity disks must be accessed to calculate the Hamming code and then write the data. Therefore, this structure may degrade the performance. Due to this reason, multiple requests cannot be serviced in this structure. Some systems therefore relax the condition of calculating the Hamming code in write operation but not on the read operation.
- The RAID level 2 is applicable where more disk errors appear. However, today, the modern disk drives come with fault-tolerance mechanisms or with built-in error-detection such as Hamming code. Therefore, this level is not used nowadays.





RAID Levels...

- RAID Level 3 (Fine-grained Striping and Single-parity Disk): This structure is also based on bit-level or byte-level striping. However, instead of Hamming code, it uses XOR ECC. The XOR code exploits the exclusive-OR operation to compute the parity of each strip. Suppose, a strip $X = \{1010\}$, the parity bit is 0 as there are even number of 1s. Suppose $X = \{1110\}$, the parity bit here is 1 as there are odd number of 1s. The parity for the i th strip on each data disk is calculated by applying XOR operations on the contents of i th strip of each disk. In case of a disk failure, the data on the disk can be reconstructed from the contents of the corresponding strips on remaining disks in the disk array by applying XOR operation on the strips. This structure uses only single-parity disk to store the parity of strips, thereby reducing the redundant storage space as shown in Fig. 15.21.
- Since the strip size is very small, the data transfer rate is high as parallel access on separate disks is possible especially in large files, but, multiple requests in parallel cannot be serviced.





RAID Levels...

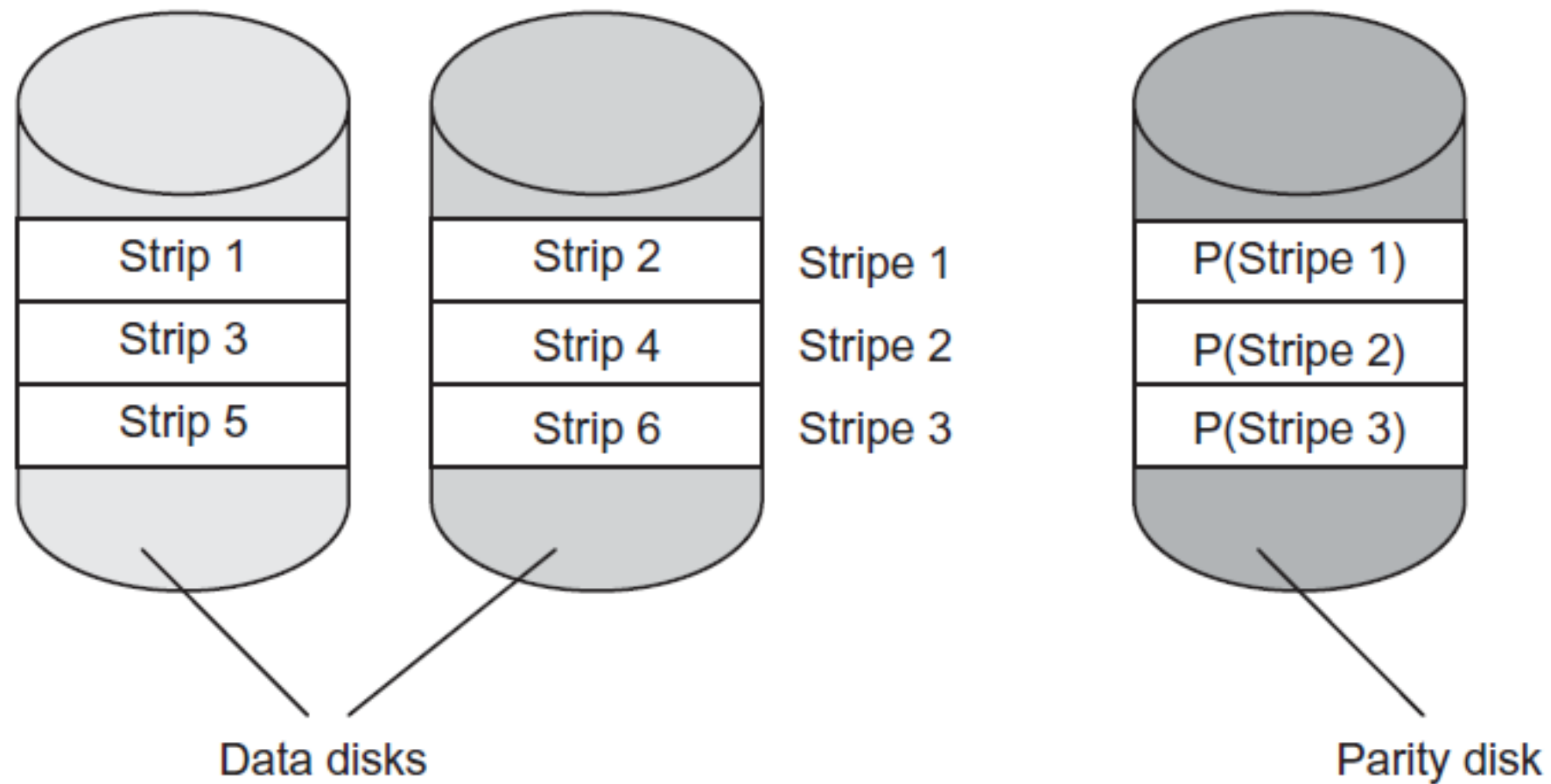


Fig. 15.21 RAID level 3





RAID Level 3: Example

- There are three data disks D1, D2, and D3, each consisting of a strip of single bit. The parity bit of i th strip is stored on the separate parity disk D4 and can be calculated as $D4(i) = D3(i) \oplus D2(i) \oplus D1(i)$.
- Suppose, disk D2 fails and the data on it is lost. In this case, the data of D2 can be regenerated as follows:
- $D2(i) = D4(i) \oplus D3(i) \oplus D1(i)$





RAID Levels...

- ❑ **RAID Level 4 (Block Level XOR ECC Parity):** This structure uses large-sized strips and the data is striped as fixed-sized blocks, generally larger than a byte. In effect, each strip stores more data, thereby allowing multiple I/O requests to be serviced in parallel.
- ❑ The drawback is that the data-transfer rate becomes slow due to coarse-grained strips. It uses exclusive-OR ECC as used in RAID level 3.
- ❑ The parity bit for the strip on each disk is calculated and stored on the parity disk. Since the parity calculation is required for all read and write operations, the parity calculation can be eliminated for multiple read requests so that they can be accessed faster.
- ❑ However, it cannot be ignored in case of write operations. The large data read or write operations are fast in this structure as parallel access is possible. However, a small data read or write access may become slow due to parity bit calculation.





RAID Levels...

- Any write operation needs to read the old data block, update it, and then write the new data block. This read-update-write cycle may affect the performance of the system. Moreover, the old parity block is read and then modified after recalculating the new parity bit.
- Thus, any write operation involves two reads for old blocks and two writes for the new blocks.
- Therefore, in case of small write requests, the overhead increases, thereby reducing the performance.
- Further, in case of multiple write requests, either small or large, the parity disk becomes a bottleneck as each write request needs to write on the disk for new parity bit.





RAID Levels...

- **RAID Level 5 (Block Level Distributed XOR ECC Parity):** In this structure, the parity bits are not stored in a single devoted disk. Rather each disk in the disk array participates to store the parity strip in a round robin fashion as shown in Fig. 15.22.

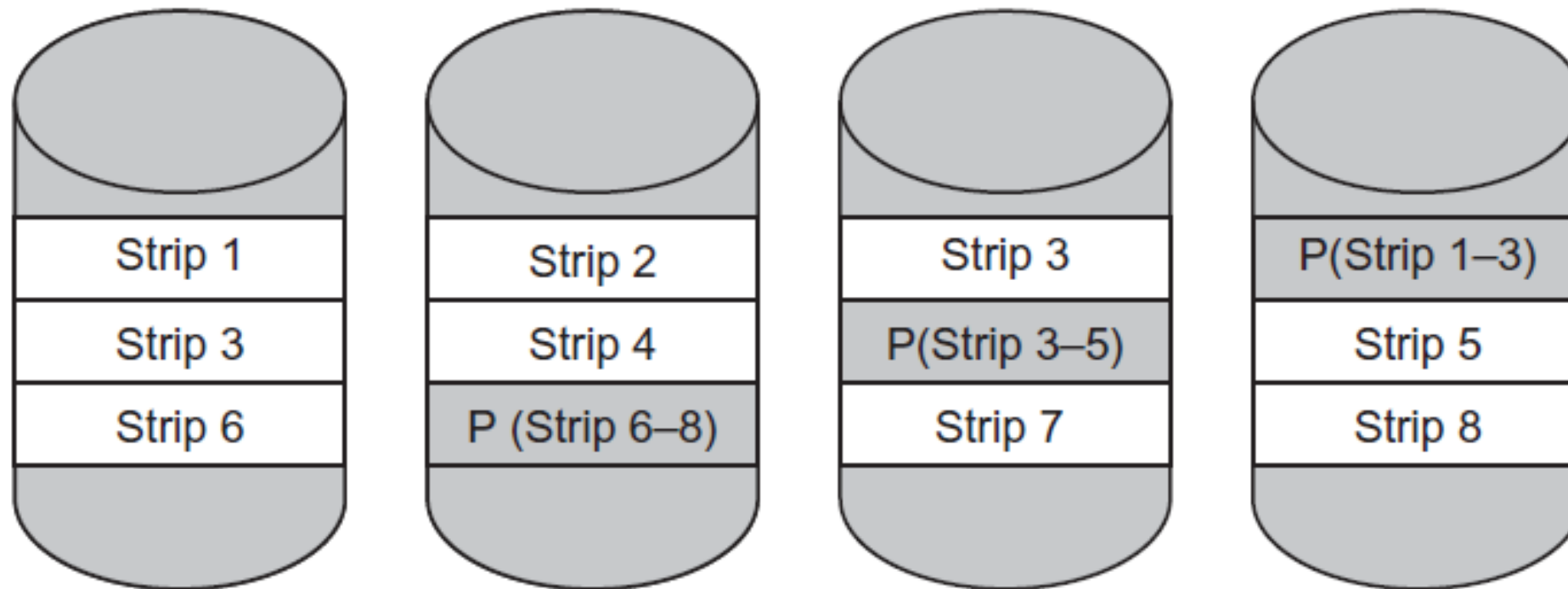


Fig. 15.22 RAID level 5





RAID Levels...

- This reduces the bottleneck seen in RAID level 4, and now the parity bits can be updated in parallel.
- However, the read-update-write cycle still exists for each write operation. There are different methods to reduce this overhead. One method is to cache the recently accessed data and parity strip. Second method is to store only the difference between the old parity and new parity bits in the memory. This is known as parity logging.
- The third method is to reduce the I/O operations during write operation to defer the parity generation in case of multiple small write operations at once and perform parity calculation only when the system load is light. This technique is known as A Frequently Redundant Array of Independent Disks (AFRAID). This structure is frequently used RAID level due to its balanced features of reliability, performance, and cost.





RAID Levels...

- ❑ **RAID Level 6 (Block Level Dual Parity):** This structure enhances the reliability by including two types of parity calculations. One of them is XOR as used in previous structures. The other calculation method is based on an algorithm that is independent of data stored on the disks, providing higher reliability on the disks failure. The two types of parity strips (say, P1 and P2) calculated are stored on two separate disks (see Fig. 15.23), thereby increasing the redundant storage.
- ❑ Thus, RAID level 6 may suffer in performance due to two parity disks for a single write operation.

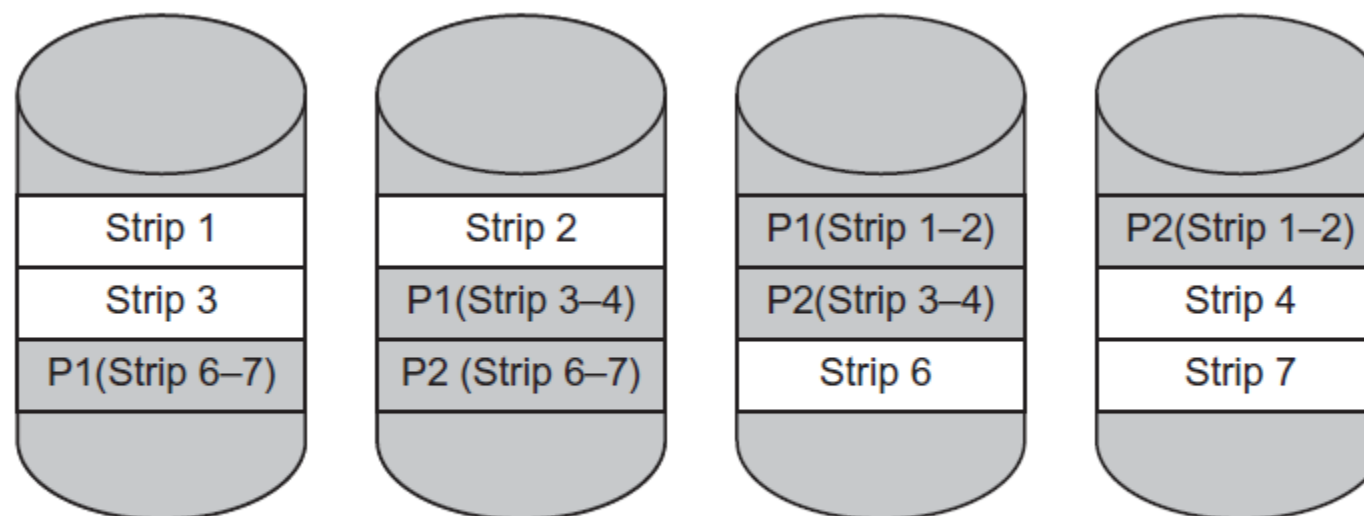


Fig. 15.23 RAID level 6





RAID Levels...

- To obtain better features of different RAID levels, RAID levels are combined.
- For example, RAID levels 0 and 1 have been combined to obtain RAID 0 + 1. It gives better performance and reliability. In this scheme, a set of n disks are striped, and then the stripe is mirrored on n redundant disks.
- The other structure that is available is RAID 1 + 0. The difference in 0 + 1 is that each data disk is first mirrored on the redundant disk and then the data disk, as well as the mirrored disk, is striped.
- The advantage in 1 + 0 is that in case of failure of a single disk, the mirror copy of the whole disk is available, while in case of 0 + 1, only mirrored strip of the failed disk is available.
- Therefore, RAID 1 + 0 provides better reliability. Other RAID levels such as 0 + 3, 0 + 5, and 1 + 5 have also been developed.



End

Unit-5 Part-1:

Disk Management

